SHADOWSTRIKE-LABS

ShadowStrike Phantom

Open-source next-generation endpoint protection for Windows. A custom kernel sensor, on-device analysis and a full-system emulation engine — built from scratch.

EVERY LINE AUDITABLE

ShadowStrike-Labs
contact@shadowstrike.dev
shadowstrike.dev

Company Pitch
August 2026
Alpha · Beta target 2027

01 THE PROBLEM

Security software demands the deepest trust and offers the least transparency.

Endpoint protection runs inside the Windows kernel. It sees every file, every process and every network flow. It can stop a machine from booting. And at every major vendor, it is code the customer is not permitted to read.

The risk is not theoretical

In July 2024 a single faulty kernel update disabled roughly 8.5 million Windows machines worldwide, grounding airlines and interrupting hospitals. The affected organisations had no ability to inspect the code that did it.

Verification is becoming mandatory

Software supply-chain requirements in regulated sectors increasingly call for provenance and auditability that closed-source kernel agents structurally cannot provide.

Open source has not filled the gap

Existing open tools are scanners, log shippers or rule engines. None is a complete kernel-to-user-mode endpoint protection platform with its own detection engine.

02 WHAT WE BUILT

Five purpose-built subsystems. No third-party detection engine.

ShadowStrike Phantom is not a wrapper around existing tools and not a proof of concept. Each layer was written from the ground up so that the entire detection path — from the kernel callback to the final verdict — is inspectable.

SUBSYSTEM	WHAT IT IS	SCALE
PhantomSensor	WDM minifilter kernel driver. Twenty detection subsystems covering syscalls, memory, process, registry, network and self-protection.	305,000 lines of C
PhantomCore	Shared user-mode detection stack: real-time protection, signature and pattern stores, anti-evasion, script scanners, self-defence.	667,000 lines of C++
PhantomEmulator	Custom x86/x64 CPU emulator for safe malware detonation. Virtual OS, ten emulated DLLs, twelve analysis engines. No hypervisor dependency.	122,000 lines of C++
PhantomHome	Consumer product layer. Forty-seven protection modules spanning banking, privacy, web, email, USB and ransomware defence.	167,000 lines of C++
PhantomCortex	On-device machine learning. Five model architectures for static, behavioural, emulation, memory and network classification.	Python training pipeline

Approximately 2.3 million lines in total across C, C++, assembly and Python, including test suites and tooling.

03 ARCHITECTURE

One detection core, three products, one kernel sensor.

USER MODE · PRODUCTS

Phantom Home
Consumer endpoint, local desktop interface

Phantom EDR
Enterprise detection and response

Phantom XDR
Fleet-wide extended detection

Management dashboard
Planned, EDR and XDR tiers

SHARED DETECTION INFRASTRUCTURE

Real-time protection
On-access orchestration, exploit and behaviour blocking

Detection stores
Signature B-tree, Aho-Corasick patterns, Bloom-filter hashes, threat intel

PhantomEmulator
Detonation, unpacking, entry-point recovery, import reconstruction

PhantomCortex
Five on-device models, ONNX inference, no cloud verdicts

ENCRYPTED KERNEL BRIDGE

AES-256-GCM over FilterConnectPort
Authenticated, per-session keyed channel between kernel and user mode

KERNEL MODE · PHANTOMSENSOR.SYS · MINIFILTER ALTITUDE 385210

File system
Pre and post I/O, ransomware heuristics

Process and thread
Injection, hollowing, hijack detection

Syscall monitor
Direct syscall, Heaven's and Hell's Gate

Memory
VAD tracking, ROP, shellcode, code caves

Self-protection
Anti-tamper, callback guard, integrity

HARDWARE AND FIRMWARE · SECURE BOOT · TPM ATTESTATION · FIRMWARE INTEGRITY

04 DIFFERENTIATION

Why building it ourselves is the product.

Owning the detection path

- No licensed third-party scanning engine, so no per-seat royalty and no opaque component inside our own kernel driver.
- Custom minifilter, custom CPU emulator, custom instruction decoder, and a clean-room fuzzy hasher with no GPL dependency.
- Detection logic, heuristics and model weights all live in the repository. A customer can read the rule that flagged their file.
- Kernel and user mode share memory-mapped type definitions, so the protocol between them is verifiable rather than assumed.

Engineering evidence

0.25
Coverity defects per thousand lines, kernel driver

550+
MITRE ATT&CK technique IDs mapped across fourteen tactics

103
Packer families identified by the unpacking engine

8
Independent original-entry-point recovery methods

The kernel driver passes Windows Driver Verifier with zero violations under the full standard and low-resource simulation suites.

05 CAPABILITY DEPTH

Detection that does not depend on knowing the sample.

Signature matching alone cannot address packed, obfuscated or evasion-aware malware. The emulation engine executes a sample inside a fully virtual environment and draws its conclusions from behaviour.

Generic unpacking

Detects the write-then-execute transition every runtime packer must perform, tracks each decryption layer with entropy analysis, recovers the original entry point through eight independent heuristics, rebuilds the import table from observed API calls, and dumps a clean executable image.

Anti-evasion

Timing, debugger, virtual-machine and environment countermeasures are emulated so that samples which check whether they are being watched proceed down their real execution path instead of a decoy.

On-device inference

Five model architectures — gradient boosting, convolutional, recurrent, feedforward and autoencoder — score static, behavioural, emulation, memory and network features locally. Detection never requires sending a customer file to a server.

> *A product whose selling point is transparency cannot outsource the part that matters. That constraint shaped every engineering decision, and it is the reason the codebase is the size it is.*

06 **MARKET AND MODEL**

Three tiers on one engine.

TIER	CUSTOMER	POSITION	COMMERCIAL MODEL
Phantom Home	Consumer and prosumer endpoints	Lightweight local protection with a desktop interface. Serves as the proving ground for the shared engine.	Free and open source, paid support
Phantom EDR	Small and mid-sized organisations	Full detection and response with centralised management and forensic collection.	Per-endpoint subscription
Phantom XDR	Enterprise and regulated sectors	Extended detection across endpoint, network and identity, with fleet-wide correlation.	Per-endpoint subscription, annual

Why the licence works commercially

AGPL-3.0 makes the code auditable while making proprietary redistribution by competitors impractical. Revenue comes from the managed backend, support and the enterprise tiers, not from withholding source.

Where transparency is worth paying for

Public sector, healthcare, finance and critical infrastructure increasingly need to demonstrate what their kernel-level software does. No closed agent can satisfy that, and it is our entry point.

What we cannot build on local hardware.

Detection runs entirely on the endpoint, so the product carries no per-customer cloud cost and sends no customer files anywhere. Our infrastructure requirement is concentrated in three places: training the models, distributing signed definitions, and building and testing a Windows kernel driver without risking a user's machine.

WORKLOAD	WHAT IT INVOLVES	REQUIREMENT
Model training	Five architectures trained over 3.5 million labelled PE samples from the EMBER 2018 and 2024 corpora, plus synthetic behavioural and emulation traces.	Accelerated GPU compute; the single dependency we cannot meet locally
Kernel driver CI	Windows build agents for WDK compilation and an isolated Driver Verifier and boot-integrity matrix. A kernel regression must never reach a user.	Ephemeral Windows build and test instances
Definition delivery	Signed signature, hash and model artefacts distributed to endpoints with delta updates and verified rollback.	Object storage with global edge distribution
Dataset custody	Versioned sample corpora, extracted feature sets and reproducible training snapshots, so any released model can be rebuilt exactly.	Encrypted, lifecycle-tiered storage
Retraining automation	Scheduled ingestion from six threat-intelligence feeds, feature extraction, evaluation gates and model promotion.	Managed batch orchestration
Artefact integrity	Custody of the signing keys that endpoints trust, with least-privilege access and an audit trail for every promotion.	Managed key custody and access control

Where the project stands today.

We would rather be precise than flattering. The platform is in alpha and under active daily development.

COMPONENT	STATE	DETAIL
PhantomSensor.sys	Complete	Twenty subsystems. Coverity 0.25 defects per thousand lines. Driver Verifier clean. Loads and runs under test signing.
PhantomCore	Complete, hardening	All module families implemented; currently in a security and reliability audit phase.
PhantomEmulator	Operational	CPU emulation, virtual OS and the full analysis suite are functional and integrated into the scan pipeline.
PhantomCortex	Four of five models	Trained and exported to ONNX. The fifth awaits retraining on the expanded corpus — the primary use of additional compute.
Phantom Home	Integration	Forty-seven modules wired to the orchestrator; end-to-end reliability work in progress ahead of beta.
EDR and XDR tiers	In development	Share the completed sensor and detection core; the management backend is the remaining work.

Public beta target: early 2027. No paying customers yet. The Home tier ships first and free, deliberately, so the engine is proven in the open before anyone is asked to pay for it.

The code with the most privilege should be the code you can read.

Endpoint security is the one category where customers are asked to grant total access on trust alone. We believe that trade is avoidable, and that demonstrating so requires building the whole thing in the open — kernel driver included.

What compute accelerates

Model training is the one dependency we cannot satisfy on local hardware. Additional capacity moves the remaining model and the retraining pipeline from constrained to continuous.

What the project offers back

A reference open-source security workload: reproducible machine-learning training, signed artefact distribution and Windows kernel CI, all publicly documented and auditable.

Contact

ShadowStrike-Labs
contact@shadowstrike.dev
shadowstrike.dev
github.com/ShadowStrike-Labs